OBJECT-ORIENTED PROGRAMMING · PROJECT REPORT

Bookstore Inventory Manager

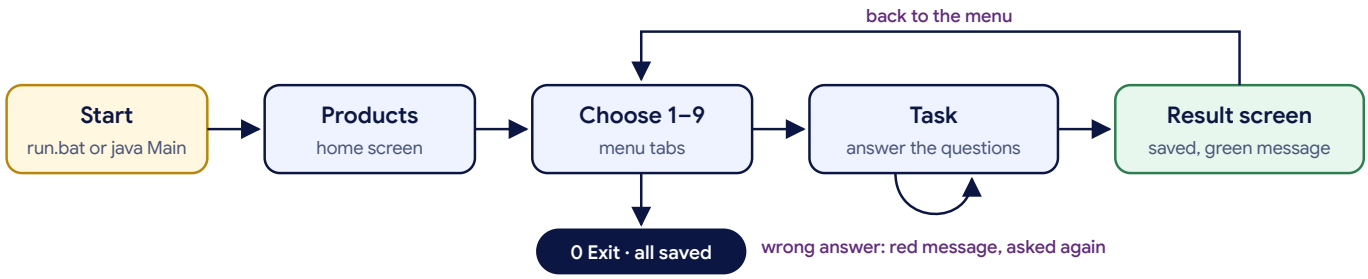
Java 17 console application · 4 files, 617 lines · github.com/AtlasRoX/Bookstore-JAVA

A program for a small bookstore that sells books and stationery. Staff can add, search and restock products, change prices, sell with an optional 5% member discount, refund sales, and see sales history and a report. Every change is saved automatically to `bookstore.dat`. A wrong answer is explained in red and asked again; typing `cancel` stops a task.

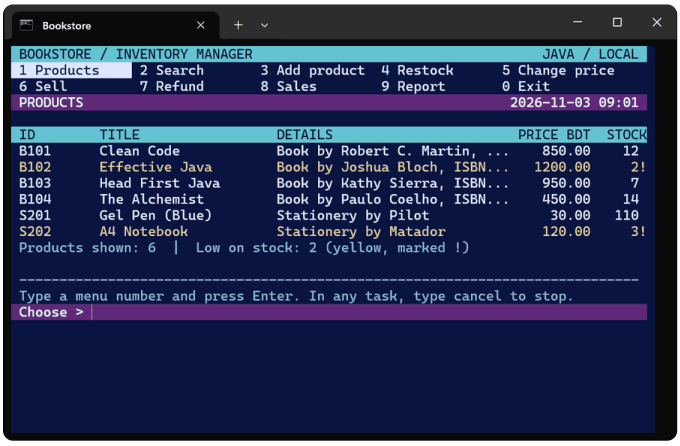
File	Job
`Product.java`	Interfaces Item, Trackable, Promotable; abstract class Product; subclasses Book and Stationery
`Bookstore.java`	Shop rules (add, restock, sell, refund, search), saving and loading; class Sale (one receipt)
`BookstoreUI.java`	Everything on screen: colors, menu tabs, tables, receipt, questions
`Main.java`	Starts the program, runs the menu and one method per task

Run: double-click `run.bat` (opens an 80 × 24 window), or `javac *.java` then `java Main`. **Tested:** 30/30 automated test cases passed; 50 random-input sessions, 0 crashes.

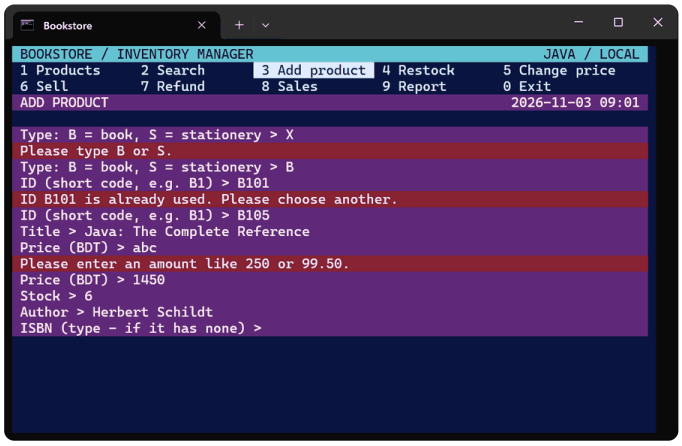
User Flow



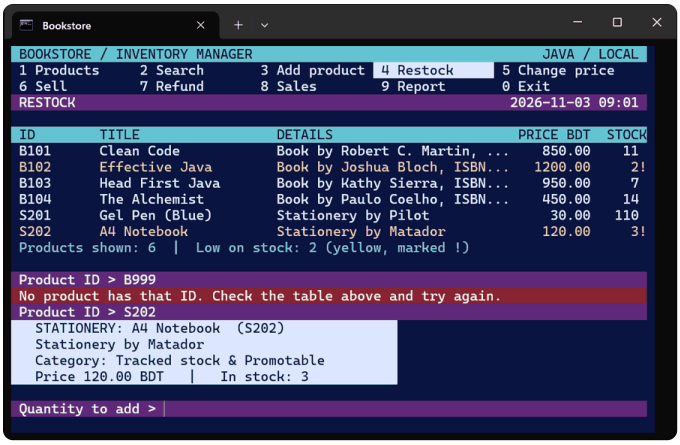
Screens



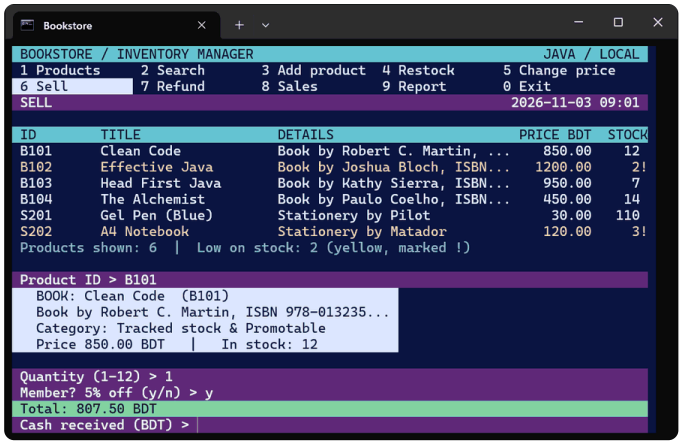
1 Products — the home screen: every product with its details, price and stock. Low stock (3 or fewer) is yellow with "!".



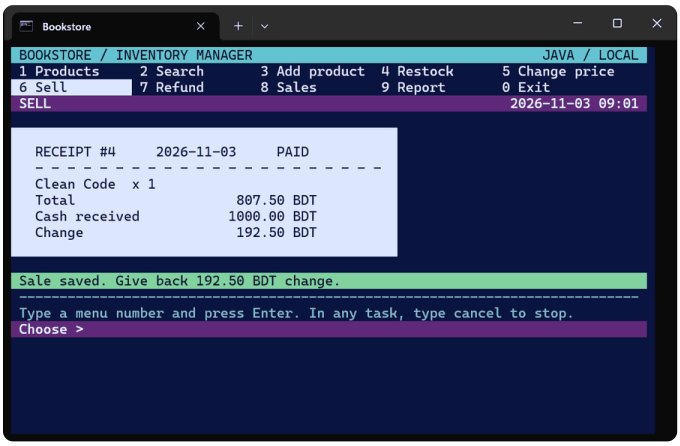
3 Add product — one question at a time; each wrong answer is explained in red and asked again.



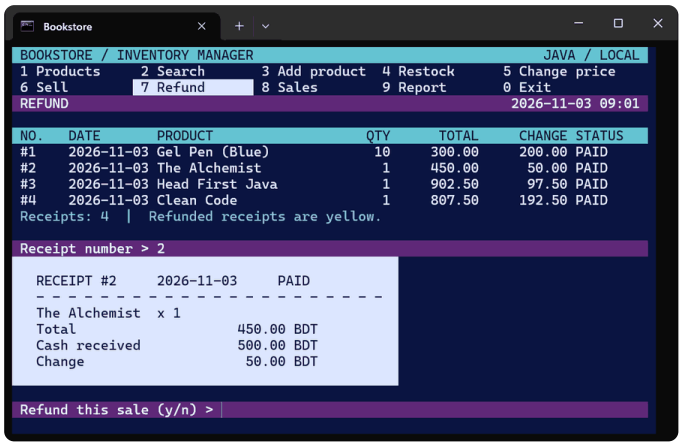
4 Restock — an unknown ID is refused; the card shows type, details, category and current stock.



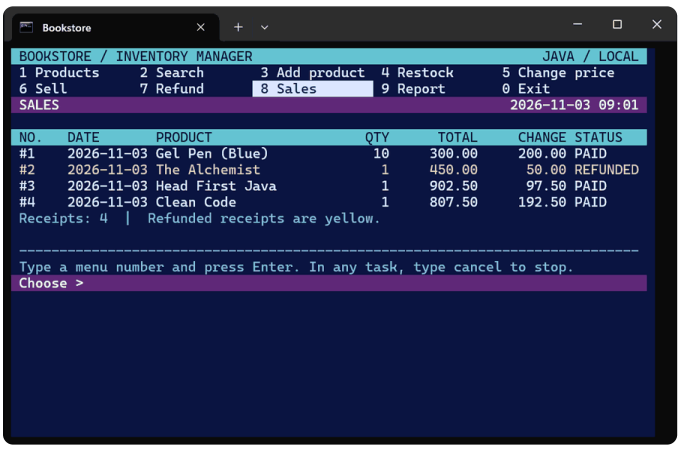
6 Sell — the quantity range is shown; a member gets 5% off; the total appears in green.



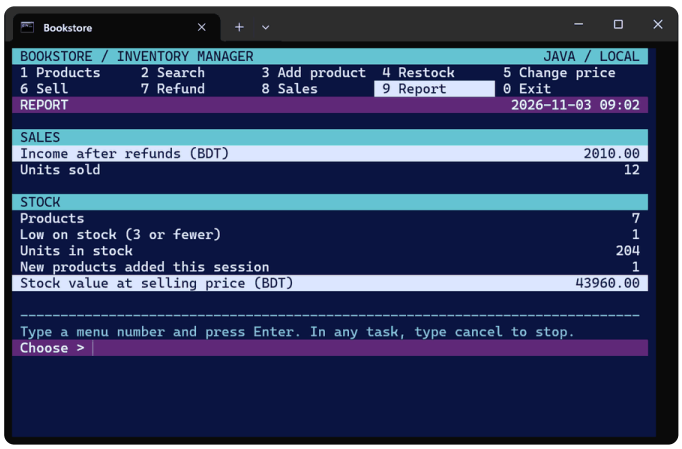
6 Sell — after payment the screen shows the receipt and the change to give back.



7 Refund — choose a receipt and confirm; the units go back into stock.



8 Sales — every receipt with total, change and status; refunded ones are yellow.



9 Report — income after refunds, units sold, stock, products added this session, stock value.

OOP Concepts Used

1 Abstraction

Abstraction means showing only what is needed and hiding the details. `Product` is declared `abstract`, so `new Product(...)` does not compile: the shop can only create a `Book` or a `Stationery`. `getType()` and `getDetails()` have no body here, so every product must be able to describe itself, but each type decides how. The tables, search and

receipts work only with `Product` and never need to know the exact type.

Product.java lines 25–83

```
25 // ABSTRACTION: Product is abstract, so the program can only make a Book or a Stationery.
26 // ENCAPSULATION: the fields are private and are used only through getters and setters.
27 public abstract class Product implements Trackable, Promotable, Serializable {
…
81 // ABSTRACT METHODS: every subclass must write its own version (overriding).
82 public abstract String getType();
83 public abstract String getDetails();
```

2 Encapsulation

Encapsulation means keeping an object's data private and changing it only through its own methods. The fields of `Product` are `private`; other classes read them with getters and change them with setters, which refuse a negative price or stock, so a product can never hold impossible values. `Bookstore` also hides its lists of products and sales, so only its own methods (add, sell, refund …) can change them, and every change is saved.

Product.java lines 39–68

```
39 // NON-STATIC (instance) variables: every product object has its own copy.
40 private String id, title;
41 private double price;
42 private int stock;
…
55 // NON-STATIC METHODS: getters and setters work on one object's own data.
56 public String getId() { return id; }
57 public String getTitle() { return title; }
58 public double getPrice() { return price; }
59 public int getStock() { return stock; }
61 // The setters protect the data: a negative value is refused.
62 public void setPrice(double price) {
63     if (price >= 0) this.price = price;
64 }
66 public void setStock(int stock) {
67     if (stock >= 0) this.stock = stock;
68 }
```

Bookstore.java lines 8–13

```
8  private static final String FILE = "bookstore.dat";
9  private ArrayList<Product> products = new ArrayList<>();
10 private ArrayList<Sale> sales = new ArrayList<>();
12 public ArrayList<Product> getProducts() { return products; }
13 public ArrayList<Sale> getSales() { return sales; }
```

3 Inheritance

Inheritance lets a subclass reuse everything from its superclass. `Book` and `Stationery` `extend` `Product`, so they get id, title, price, stock, all getters and setters and `calculateDiscount()` without writing them again. Each subclass only adds what is special about it: a book has an author and an ISBN, a stationery item has a brand. Because both are a `Product`, the inventory and every screen treat them the same way.

Product.java lines 86–123

```
86  // INHERITANCE: Book is a subclass of the superclass Product.
87  class Book extends Product {
88      private static final long serialVersionUID = 1L;
89      private String author, isbn;
…
103 public String getAuthor() { return author; } // only a Book has an author
…
113 // INHERITANCE: Stationery is another subclass of Product.
114 class Stationery extends Product {
115     private static final long serialVersionUID = 1L;
116     private String brand;
…
123 public String getBrand() { return brand; } // only Stationery has a brand
```

4 Polymorphism

Polymorphism means one call, many forms. In `showProducts` every item is held in a `Product` variable, yet `p.getDetails()` runs Book's version ("Book by …, ISBN …") for books and Stationery's version ("Stationery by …") for stationery: Java chooses the method of the real object while the program runs (runtime polymorphism). The product card

does the same with `p.getType()`. Method overloading (6) is the compile-time form of polymorphism.

BookstoreUI.java lines 131–162

```
131 // A table of products. Low stock (3 or fewer) is yellow and marked with "!".
132 public void showProducts(ArrayList<Product> list) {
133     line(CYAN, String.format(" %-9s %-21s %-29s %9s %6s", "ID", "TITLE", "DETAILS", "PRICE BDT",
    "STOCK"));
134     int low = 0;
135     for (Product p : list) {
136         String color = BODY;
137         String stock = p.getStock() + " ";
138         if (p.getStock() <= 3) {
139             color = YELLOW;
140             stock = p.getStock() + "!";
141             low++;
142         }
143         line(color, String.format(" %-9s %-21s %-29s %9.2f %6s",
144                 fit(p.getId(), 9), fit(p.getTitle(), 21), fit(p.getDetails(), 29), p.getPrice(),
    stock));
145     }
146     if (list.isEmpty()) line(DIM, " No products found. Choose 3 to add one.");
147     line(DIM, " Products shown: " + list.size() + "  |  Low on stock: " + low + " (yellow, marked !)");
148     line(BODY, "");
149 }
…
156 public void showProduct(Product p) {
157     card(p.getType() + ": " + p.getTitle() + "  (" + p.getId() + ")");
158     card(p.getDetails());
159     card("Category: " + p.getCategory());
160     card(String.format("Price %.2f %s   |   In stock: %d", p.getPrice(), Product.CURRENCY,
    p.getStock()));
161     line(BODY, "");
162 }
```

5 Static vs non-static

A `static` member belongs to the class and is shared by all objects; a non-static member belongs to each object. `CURRENCY` is one shared text, set once in a `static` block when Java loads the class. `createdCount` is a shared counter that every constructor call increases; the static method `Product.getCreatedCount()` reads it and the Report shows it as "New products added this session". `price` and `stock` are non-static: every product has its own. `Main` uses only static members, so no `Main` object is ever created.

Product.java lines 30–53

```
30 // STATIC variables belong to the class and are shared by every product.
31 public static final String CURRENCY;
32 private static int createdCount = 0; // products created while the program runs (shown in the Report)
34 // STATIC BLOCK: runs once, when Java first loads the Product class.
35 static {
36     CURRENCY = "BDT";
37 }
…
49 createdCount++;
…
52 // STATIC METHOD: called on the class itself, e.g. Product.getCreatedCount()
53 public static int getCreatedCount() { return createdCount; }
```

BookstoreUI.java lines 160–212

```
160 card(String.format("Price %.2f %s   |   In stock: %d", p.getPrice(), Product.CURRENCY, p.getStock()));
…
212 line(BODY, String.format(" %-58s%19d", "New products added this session", Product.getCreatedCount()));
```

Main.java lines 4–6

```
4 public class Main {
5     static BookstoreUI ui = new BookstoreUI();
6     static Bookstore store;
```

6 Method overloading

Method overloading means several methods or constructors with the same name but different parameters; the compiler picks the right one from the arguments. `Book` has two constructors: with an ISBN, and without one (many local books have none, so the user types "-"). `Bookstore` has two `restock` methods: the Restock screen calls `restock(String, int)`

with the product ID, which finds the product and calls `restock(Product, int)` to do the work.

Product.java lines 91–99

```
91  // CONSTRUCTOR OVERLOADING 1: a book with an ISBN.
92  public Book(String id, String title, double price, int stock, String author, String isbn) {
…
98  // CONSTRUCTOR OVERLOADING 2: a book without an ISBN. THIS(...) calls the constructor above.
99  public Book(String id, String title, double price, int stock, String author) {
```

Bookstore.java lines 45–54

```
45  // METHOD OVERLOADING: restock by product ID, or by the product object itself.
46  public void restock(String id, int quantity) {
47      restock(get(id), quantity);
48  }
50  public void restock(Product p, int quantity) {
51      if (quantity < 1) throw new RuntimeException("Quantity must be at least 1.");
52      p.setStock(p.getStock() + quantity);
53      save();
54  }
```

Main.java lines 77–96

```
77  String isbn = ui.askText("ISBN (type - if it has none)");
78  Book book; // SUBCLASS reference
79  if (isbn.equals("-")) book = new Book(id, title, price, stock, author); // overloaded constructor
80  else book = new Book(id, title, price, stock, author, isbn);
…
96  store.restock(p.getId(), ui.askNumber("Quantity to add", 1, 1000000)); // overloaded restock(String,
    int)
```

7 Method overriding

Overriding means a subclass writes its own version of a method it received from its superclass or an interface. `Product` only declares `getType()` and `getDetails()`; `Book` and `Stationery` each give their own body. `@Override` asks the compiler to check that the method really exists in the parent, so a spelling mistake becomes a compile error. These overrides are what make the polymorphism in (4) work.

Product.java lines 105–129

```
105 // METHOD OVERRIDING: Book's own versions of Product's abstract methods.
106 @Override
107 public String getType() { return "BOOK"; }
109 @Override
110 public String getDetails() { return "Book by " + author + ", ISBN " + isbn; }
…
125 @Override
126 public String getType() { return "STATIONERY"; }
128 @Override
129 public String getDetails() { return "Stationery by " + brand; }
```

8 The this keyword

`this` means “the current object”. In `this.id = id` the left side is the object’s own field and the right side is the constructor parameter; without `this` the parameter would hide the field. `this(...)` calls another constructor of the same class: the Book constructor without an ISBN passes its values to the full constructor and fills the ISBN with “N/A”, so the setup code is written only once. The `Sale` constructor in `Bookstore.java` uses `this` the same way.

Product.java lines 44–101

```
44  public Product(String id, String title, double price, int stock) {
45      this.id = id; // THIS: this.id is the object's field, id is the parameter
46      this.title = title;
47      this.price = price;
48      this.stock = stock;
49      createdCount++;
50  }
…
98  // CONSTRUCTOR OVERLOADING 2: a book without an ISBN. THIS(...) calls the constructor above.
99  public Book(String id, String title, double price, int stock, String author) {
100     this(id, title, price, stock, author, "N/A");
101 }
```

9 The super keyword

`super` refers to the parent class. A subclass constructor starts with `super(id, title, price, stock)`, which runs the `Product` constructor first: it stores the shared fields and increases the static counter. Only then does the subclass store its own fields (author and ISBN, or brand). The other form, `Trackable.super.getCategory()`, calls a parent interface's method; see (11).

Product.java lines 91–121

```
91  // CONSTRUCTOR OVERLOADING 1: a book with an ISBN.
92  public Book(String id, String title, double price, int stock, String author, String isbn) {
93      super(id, title, price, stock); // SUPER: runs the Product constructor first
94      this.author = author;
95      this.isbn = isbn;
96  }
…
118 public Stationery(String id, String title, double price, int stock, String brand) {
119     super(id, title, price, stock); // SUPER: runs the Product constructor first
120     this.brand = brand;
121 }
```

10 Subclass vs superclass reference

A subclass reference (`Book book`) can use everything a Book has, including `getAuthor()`. A superclass reference (`Product product`) can point to any subclass object (upcasting) but only sees the methods declared in `Product`. In Add product the new object is first held as a `Book` or `Stationery` to read its author or brand; then `product = book` stores it as a `Product`. From then on every item is treated the same: `store.add(product)` puts it in the `ArrayList<Product>`, and `product.getType()` still runs Book's own version.

Main.java lines 63–90

```
63  static void addProduct() {
64      char type = ui.askLetter("Type: B = book, S = stationery", "BS");
65      String id = ui.askText("ID (short code, e.g. B1)");
66      while (store.find(id) != null) {
67          ui.error("ID " + id + " is already used. Please choose another.");
68          id = ui.askText("ID (short code, e.g. B1)");
69      }
70      String title = ui.askText("Title");
71      double price = ui.askMoney("Price (BDT)", 0);
72      int stock = ui.askNumber("Stock", 0, 1000000);
73      Product product; // SUPERCLASS reference: can hold a Book or a Stationery
74      String extra;
75      if (type == 'B') {
76          String author = ui.askText("Author");
77          String isbn = ui.askText("ISBN (type - if it has none)");
78          Book book; // SUBCLASS reference
79          if (isbn.equals("-")) book = new Book(id, title, price, stock, author); // overloaded
    constructor
80          else book = new Book(id, title, price, stock, author, isbn);
81          extra = " by " + book.getAuthor(); // getAuthor() works only through the Book reference
82          product = book;                    // UPCASTING: the Book is stored as a Product
83      } else {
84          Stationery item = new Stationery(id, title, price, stock, ui.askText("Brand"));
85          extra = " from " + item.getBrand();
86          product = item;
87      }
88      store.add(product);
89      showResult(3, "Added " + product.getType() + " \"" + title + "\"" + extra + ".");
90  }
```

11 Diamond problem resolution

Java does not allow `class C extends A, B`, which avoids the diamond problem for classes. With interfaces it can still happen: `Trackable` and `Promotable` both extend `Item` and both give their own `default getCategory()`. `Product` implements both, so Java cannot know which one to use and refuses to compile until `Product` overrides the method. It chooses explicitly with `Trackable.super.getCategory()` and `Promotable.super.getCategory()`; the result, "Tracked

stock & Promotable", is shown on the product card (Restock and Sell screenshots).

Product.java lines 11–23

```java
interface Item {
    default String getCategory() { return "Item"; }
}
interface Trackable extends Item {
    @Override
    default String getCategory() { return "Tracked stock"; }
}
interface Promotable extends Item {
    @Override
    default String getCategory() { return "Promotable"; }
}
```

Product.java lines 27–79

```java
public abstract class Product implements Trackable, Promotable, Serializable {
…
// DIAMOND PROBLEM SOLVED: we say exactly which parent versions to use.
@Override
public String getCategory() {
    return Trackable.super.getCategory() + " & " + Promotable.super.getCategory();
}
```

BookstoreUI.java line 159

```java
card("Category: " + p.getCategory());
```

Behind the Screens

A Saving the data

All products and sales live inside one `Bookstore` object. Because the classes implement `Serializable`, `save()` writes the whole object to `bookstore.dat` in one step after every change, and `load()` reads it back when the program starts. If the file is damaged, the program says so and never overwrites it.

Bookstore.java lines 93–110

```java
// Saves the whole bookstore (products and sales) to the file in one step.
private void save() {
    try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(FILE))) {
        out.writeObject(this);
    } catch (IOException e) {
        throw new RuntimeException("Could not save: " + e.getMessage());
    }
}
// Reads the saved bookstore, or starts an empty one the first time.
public static Bookstore load() {
    if (!new File(FILE).exists()) return new Bookstore();
    try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(FILE))) {
        return (Bookstore) in.readObject();
    } catch (Exception e) {
        throw new RuntimeException("Could not read " + FILE + ": " + e.getMessage());
    }
}
```

B Checking every answer

Every question goes through `askText`: it draws the purple input bar, ignores empty answers and turns `cancel` into an exception that ends the task. `askNumber` (and `askMoney`) keep asking until the answer is a number in the allowed range,

so a mistake never cancels the whole task.

BookstoreUI.java lines 79–103

```java
// Keeps asking until something is typed. Typing "cancel" stops the current task.
public String askText(String question) {
    while (true) {
        // Draw a purple bar, then \r jumps back so the typing starts right after the question.
        System.out.print(PURPLE + String.format(" %-78s", question + " >") + "\r " + question + " > ");
        if (!in.hasNextLine()) System.exit(0); // input was closed
        String answer = in.nextLine().trim();
        System.out.print(BODY);
        if (answer.equalsIgnoreCase("cancel")) throw new RuntimeException("Cancelled. Nothing
changed.");
        if (!answer.isEmpty()) return answer;
    }
}
// Asks again until the answer is a whole number from min to max.
public int askNumber(String question, int min, int max) {
    while (true) {
        try {
            int number = Integer.parseInt(askText(question));
            if (number >= min && number <= max) return number;
        } catch (NumberFormatException e) {
            // not a number, so ask again
        }
        error("Please enter a whole number from " + min + " to " + max + ".");
    }
}
```

C Shop rules and errors

`Bookstore` checks its rules again before changing any data: the quantity must be in stock, the cash must cover the total, and members get 5% off through `calculateDiscount(5)`. A broken rule throws a `RuntimeException`; the menu loop in `Main` catches it, shows it in a red bar and returns to the menu, so the program never crashes.

Bookstore.java lines 61–68

```java
// Members get 5% off. The total is rounded to 2 decimal places.
public double total(Product p, int quantity, boolean member) {
    if (quantity < 1 || quantity > p.getStock())
        throw new RuntimeException("Quantity must be from 1 to " + p.getStock() + ".");
    double total = p.getPrice() * quantity;
    if (member) total = total - p.calculateDiscount(5) * quantity;
    return Math.round(total * 100) / 100.0;
}
```

Main.java lines 34–36

```java
} catch (RuntimeException e) {
    ui.error(e.getMessage()); // show the problem in red, then go back to the menu
}
```